

- Client and server will now derive the symmetric key KSymm2 using the D-H algorithm. The D-H algorithm does not require the symmetric key to be exchanged, but it derives it without sharing the actual symmetric key. So, the client and server will derive the symmetric key KSymm2.
- To exchange the generator and the prime number as a part of D-H implementation, the client and server will use KPriv2 and KPub2, and KPriv3 and KPub3 as follows:
 - Server will encrypt the prime number and generator number with client, using its private key KPriv3.
 - Client will extract the numbers using server's temporary public key as KPub3.
 - Client and server will generate intermediate numbers using the D-H algorithm, which we will term as INum1 and INum2. Client and server will use some secret number at their own ends [secret-client and secret-server] and will derive INum1 and INum2.
 - Client and server will not share these secret numbers with each other.
 - The client will encrypt the intermediate number INum1 with KPub3, and will send the encrypted number to the server. Since the server has KPriv3 corresponding to KPub3, only the server can decrypt the number and can extract INum1.
 - Server will encrypt INum2 using the client's public key KPub2, and will share the encrypted number with the client. Since only the client has the corresponding private key, KPriv2, only it can decrypt the number and extract it.
 - Now the client has INum2 and server has INum1. The client has its secret number [secret-client] and server has its own secret number [secret-server]. Using their secrets and the D-H known algorithm, both the client and server will derive the symmetric key KSymm2. This KSymm2 will not be transmitted

over the wire, so an attacker will not be able to get hold of it.

- Server and client will discard the public and private keys as KPub2, KPriv2, KPriv3 and KPub3 once the KSymm2 is derived.
- Now both the client and server have KSymm2, which is used for encrypting the traffic between them.
- At the end of the session, the symmetric key KSymm2 is discarded by both the server and the client.

Detailed analysis of the alternate solution

The solution discussed here uses two sets of public-private keys. These keys [KPub2-KPriv2 and KPub3-KPriv3] are temporary and live for only the duration of the session. Due to the short-lived nature of these keys, an attacker gets very less time to break them.

Additionally, an attacker will have to break two private keys in a short time span. This seems very unlikely and reduces the possibility of getting to the symmetric key [session key] to be used to interpret the communication.

What an attacker will have in case a server's long-lived key is compromised

KPub1: Server's long-lived public key, which can be taken from the server's certificate

KPriv1: Server's long-lived private key [case of compromised key]

KSymm1: Session key which an attacker can decrypt using KPriv1

KPub2: Client's temporary public key
KPub3: Server's temporary public key

What an attacker will NOT have in case a server's long-lived key is compromised

KPriv2: Client's temporary private key

KPriv3: Server's temporary private key
INum1 and INum2: Intermediate numbers derived as part of Diffie-Hellman

Secrets: Secret numbers at client and server. These are never shared.

KSymm2: Symmetric key derived by both server and client. This is never shared but derived. An attacker will not get this as it does not have short-lived private keys of server and client.

There are chances of a server's long-lived private key being compromised, and hence it needs to be either revoked or refreshed after a certain duration. The lesser the duration, the lesser are the chances of its compromise. Due to the complexities involved in explicit revocation of long-lived keys, they are not revoked frequently.

Additionally, the revocation does not address the issue of the duration during which the compromise of the long-lived private key went undetected. So, explicit revocation is a post detection procedure, while the solution given here adds value in securing the communication even in the case of a compromise of the private key. Even if the server's long-lived private key [KPriv1] is compromised, the alternate solution discussed here ensures the communication is secure. **END** 🐧

Note: This article was originally published in *PenTest Magazine* at <https://pentestmag.com/product/pentest-build-your-own-pentest-lab-in-2021/>

References

- <https://www.digicert.com/faq/ssl-cryptography.htm>
- <https://doubleoctopus.com/security-wiki/encryption-and-cryptography/diffie-hellman-algorithm/>
- <https://escholarship.org/uc/item/0pm199fp>

By: Sanjay Phanshikar

The author is a technical lead at IBM ISL, and is into information security. The focus of his work includes Web application endpoint security.